02/05/2025

AI-Powered Translation Web Application

Project Report

Bellatreche Mohamed Amine
USTO ING 3 SD

Table des matières

1. Introduction

This report details the development process of an AI-powered web application called Tarjama, designed for translating text and documents between various languages and Arabic (Modern Standard Arabic - Fusha). The application features a RESTful API backend built with FastAPI and a user-friendly frontend using HTML, CSS, and JavaScript. It is designed for deployment on Hugging Face Spaces using Docker.

2. Project Objectives

- Develop a functional web application with AI translation capabilities.
- Deploy the application on Hugging Face Spaces using Docker.
- Build a RESTful API backend using FastAPI.
- Integrate Hugging Face LLMs/models for translation.
- Create a user-friendly frontend for interacting with the API.
- Support translation for direct text input and uploaded documents (PDF, DOCX, TXT).
- Focus on high-quality Arabic translation, emphasizing meaning and eloquence (Balagha) over literal translation.
- Implement a robust fallback mechanism to ensure translation service availability.
- Support language switching and reverse translation capability.
- Enable downloading of translated documents in various formats.
- Include quick phrase features for common expressions.
- Document the development process comprehensively.

3. Backend Architecture and API Design

3.1. Framework and Language

- **Framework:** FastAPI
- **Language:** Python 3.9+

3.2. Directory Structure

```
/
|-- backend/
|   |-- Dockerfile
|   |-- main.py         # FastAPI application logic, API endpoints
|   |-- requirements.txt # Python dependencies
|-- static/
|   |-- script.js       # Frontend JavaScript
|   |-- style.css       # Frontend CSS
|-- templates/
|   |-- index.html      # Frontend HTML structure
|-- uploads/            # Temporary storage for uploaded files (created by
app)
|-- project_report.md   # This report
|-- deployment_guide.md # Deployment instructions
```

```
|-- project_details.txt # Original project requirements
|-- README.md           # For Hugging Face Space configuration
```

3.3. API Endpoints

- **`GET /`**
  - **Description:** Serves the main HTML frontend page (`index.html`).
  - **Response:** `HTMLResponse` containing the rendered HTML.
- **`GET /api/languages`**
  - **Description:** Returns the list of supported languages.
  - **Response:** `JSONResponse` with a mapping of language codes to language names.
- **`POST /translate/text`**
  - **Description:** Translates a snippet of text provided in the request body.
  - **Request Body:**
    - `text` (str): The text to translate.
    - `source_lang` (str): The source language code (e.g., 'en', 'fr', 'ar'). 'auto' is supported for language detection.
    - `target_lang` (str): The target language code (e.g., 'ar', 'en').
  - **Response (`JSONResponse`):**
    - `translated_text` (str): The translated text.
    - `detected_source_lang` (str, optional): The detected source language if 'auto' was used.
    - `success` (bool): Indicates if the translation was successful.
  - **Error Responses:** `400 Bad Request` (e.g., missing text), `500 Internal Server Error` (translation failure).
- **`POST /translate/document`**
  - **Description:** Uploads a document, extracts its text, and translates it.
  - **Request Body (Multipart Form Data):**
    - `file` (UploadFile): The document file (.pdf, .docx, .txt).
    - `source_lang` (str): Source language code or 'auto' for detection.
    - `target_lang` (str): Target language code.
  - **Response (`JSONResponse`):**
    - `original_filename` (str): The name of the uploaded file.
    - `original_text` (str): The extracted text from the document.
    - `translated_text` (str): The translated text.
    - `detected_source_lang` (str, optional): The detected source language if 'auto' was used.
    - `success` (bool): Indicates if the translation was successful.
  - **Error Responses:** `400 Bad Request` (e.g., no file, unsupported file type), `500 Internal Server Error` (extraction or translation failure), `501 Not Implemented` (if required libraries missing).
- **`POST /download/translated-document`**

- **Description:** Creates a downloadable version of the translated document in various formats.
- **Request Body:**
  - `content` (str): The translated text content.
  - `filename` (str): The desired filename for the download.
  - `original_type` (str): The original file's MIME type.
- **Response:** Binary file data with appropriate Content-Disposition header for download.
- **Error Responses:** `400 Bad Request` (missing parameters), `500 Internal Server Error` (document creation failure), `501 Not Implemented` (if required libraries missing).

3.4. Dependencies

Key Python libraries used:

- `fastapi`: Web framework.
- `uvicorn[standard]`: ASGI server.
- `python-multipart`: For handling form data (file uploads).
- `jinja2`: For HTML templating.
- `transformers[torch]`: For interacting with Hugging Face models.
- `torch`: Backend for `transformers`.
- `tensorflow`: Alternative backend for model acceleration.
- `googletrans`: Google Translate API wrapper (used in fallback mechanism).
- `PyMuPDF`: For PDF text extraction and creation.
- `python-docx`: For DOCX text extraction and creation.
- `langdetect`: For automatic language detection.
- `sacremoses`: For tokenization with MarianMT models.
- `sentencepiece`: For model tokenization.
- `accelerate`: For optimizing model performance.
- `requests`: For HTTP requests to external translation APIs.

3.5. Translation Model Architecture

3.5.1. Primary Translation Models

The application implements a multi-model approach using Helsinki-NLP's opus-mt models:

```
translation_models: Dict[str, Dict] = {
    "en-ar": {
        "model": None,
        "tokenizer": None,
        "translator": None,
        "model_name": "Helsinki-NLP/opus-mt-en-ar",
    },
    "ar-en": {
```

```
        "model": None,
        "tokenizer": None,
        "translator": None,
        "model_name": "Helsinki-NLP/opus-mt-ar-en",
    },
    "en-fr": {
        "model": None,
        "tokenizer": None,
        "translator": None,
        "model_name": "Helsinki-NLP/opus-mt-en-fr",
    },
    // Additional language pairs...
}
```

- **Dynamic Model Loading**: Models are loaded on-demand based on requested language pairs.
- **Memory Management**: The application intelligently manages model memory usage, ensuring that only necessary models are loaded.
- **Restart Resilience**: Includes functionality to detect and reinitialize models if they enter a bad state.

3.5.2. Multi-Tier Fallback System

A robust multi-tier fallback system ensures translation service reliability:

1. **Primary Models**: Helsinki-NLP opus-mt models for direct translation between language pairs.
2. **Fallback System**:
   - **Google Translate API**: First fallback using the googletrans library.
   - **LibreTranslate API**: Second fallback with multiple server endpoints for redundancy.
   - **MyMemory Translation API**: Third fallback for additional reliability.

This approach ensures high availability of translation services even if individual services experience issues.

3.5.3. Language Detection

Automatic language detection is implemented using:

1. **Primary Detection**: Uses the `langdetect` library for accurate language identification.
2. **Fallback Detection**: Custom character-based heuristics analyze Unicode character ranges to identify languages like Arabic, Chinese, Japanese, Russian, and Hebrew when the primary detection fails.

3.6. Cultural Adaptation

The system implements post-processing for culturally sensitive translations:

```python
def culturally_adapt_arabic(text: str) -> str:
    """Apply post-processing rules to enhance Arabic translation with
cultural sensitivity."""
    # Replace Latin punctuation with Arabic ones
    text = text.replace('?', '؟').replace(';', '؛').replace(',', '،')

    # Remove common translation artifacts/prefixes
    common_prefixes = [
        "الترجمة:", "ترجمة:", "المترجم النص:",
        "Translation:", "Arabic translation:"
    ]
    for prefix in common_prefixes:
        if text.startswith(prefix):
            text = text[len(prefix):].strip()

    return text
```

This function ensures: - Proper Arabic punctuation replaces Latin equivalents - Common translation artifacts and prefixes are removed - The output follows Arabic writing conventions

3.7. Document Processing

Text extraction from various file formats is handled through specialized libraries:

```python
async def extract_text_from_file(file: UploadFile) -> str:
    """Extracts text content from uploaded files without writing to disk."""
    content = await file.read()
    file_extension = os.path.splitext(file.filename)[1].lower()

    if file_extension == '.txt':
        # Handle text files with encoding detection
        extracted_text = decode_with_multiple_encodings(content)
    elif file_extension == '.docx':
        # Extract text from Word documents
        doc = docx.Document(BytesIO(content))
        extracted_text = '\n'.join([para.text for para in doc.paragraphs])
    elif file_extension == '.pdf':
        # Extract text from PDF files
        doc = fitz.open(stream=BytesIO(content), filetype="pdf")
        extracted_text = "\n".join([page.get_text() for page in doc])
        doc.close()
```

Document generation for download is similarly handled through specialized functions for each format:

- **PDF**: Uses PyMuPDF (fitz) to create PDF files with the translated text
- **DOCX**: Uses python-docx to create Word documents with the translated text
- **TXT**: Simple text file creation with appropriate encoding

4. Prompt Engineering and Translation Quality Control

4.1. Desired Translation Characteristics

The core requirement is to translate *from* a source language *to* Arabic (MSA Fusha) with a focus on meaning and eloquence (Balagha), avoiding overly literal translations. These goals typically fall under the umbrella of prompt engineering when using general large language models.

4.2. Translation Model Selection and Approach

While the Helsinki-NLP opus-mt models serve as the primary translation engine, prompt engineering was explored using the FLAN-T5 model:

- **Instruction Design**: Explicit instructions were crafted to guide the model toward eloquent Arabic (Balagha) translation rather than literal translation.
- **Cultural Adaptation Prompts**: The prompts include specific guidance for cultural adaptation, ensuring that idioms, cultural references, and contextual meanings are appropriately handled in the target language.

```
def create_translation_prompt(text, source_lang, target_lang="Arabic"):
    """Create a prompt that emphasizes eloquence and cultural adaptation."""
    source_lang_name = LANGUAGE_MAP.get(source_lang, "Unknown")

    prompt = f"""Translate the following {source_lang_name} text into Modern
Standard Arabic (Fusha).
Focus on conveying the meaning elegantly using proper Balagha (Arabic
eloquence).
Adapt any cultural references or idioms appropriately rather than translating
literally.
Ensure the translation reads naturally to a native Arabic speaker.

Text to translate:
{text}

Arabic translation:"""

    return prompt
```

This prompt explicitly instructs the model to:

- Use Modern Standard Arabic (Fusha) as the target language register
- Emphasize eloquence (Balagha) in the translation style
- Handle cultural references and idioms appropriately for an Arabic audience
- Prioritize natural-sounding output over literal translation

4.3. Generation Parameter Optimization

To further improve translation quality, the model's generation parameters have been fine-tuned:

```
outputs = model.generate(
    **inputs,
    max_length=512,     # Sufficient length for most translations
    num_beams=5,        # Wider beam search for better quality
    length_penalty=1.0, # Slightly favor longer, more complete translations
    top_k=50,           # Consider diverse word choices
    top_p=0.95,         # Focus on high-probability tokens for coherence
    early_stopping=True
)
```

These parameters work together to encourage:

- More natural-sounding translations through beam search
- Better handling of nuanced expressions
- Appropriate length for preserving meaning
- Balance between creativity and accuracy

4.4. Multi-Language Support

The system supports multiple source languages through a language mapping system that converts ISO language codes to full language names for better model comprehension:

```
language_map = {
    "en": "English",
    "fr": "French",
    "es": "Spanish",
    "de": "German",
    "zh": "Chinese",
    "ru": "Russian",
    "ja": "Japanese",
    "hi": "Hindi",
    "pt": "Portuguese",
    "tr": "Turkish",
    "ko": "Korean",
    "it": "Italian"
    # Additional languages can be added as needed
}
```

Using full language names in the prompt (e.g., "Translate the following French text…") helps the model better understand the translation task compared to using language codes.

4.5. Cultural Sensitivity Enhancement

While automated translations can be technically accurate, ensuring cultural sensitivity requires special attention. The prompt engineering approach implements several strategies:

1. **Explicit Cultural Adaptation Instructions**: The prompts specifically instruct the model to adapt cultural references appropriately for the target audience.

2. **Context-Aware Translation**: The instructions emphasize conveying meaning over literal translation, allowing the model to adjust idioms and expressions for cultural relevance.

3. **Preservation of Intent**: By focusing on eloquence (Balagha), the model is guided to maintain the original text's tone, formality level, and communicative intent while adapting it linguistically.

5. Frontend Design and User Experience

5.1. Design Choices

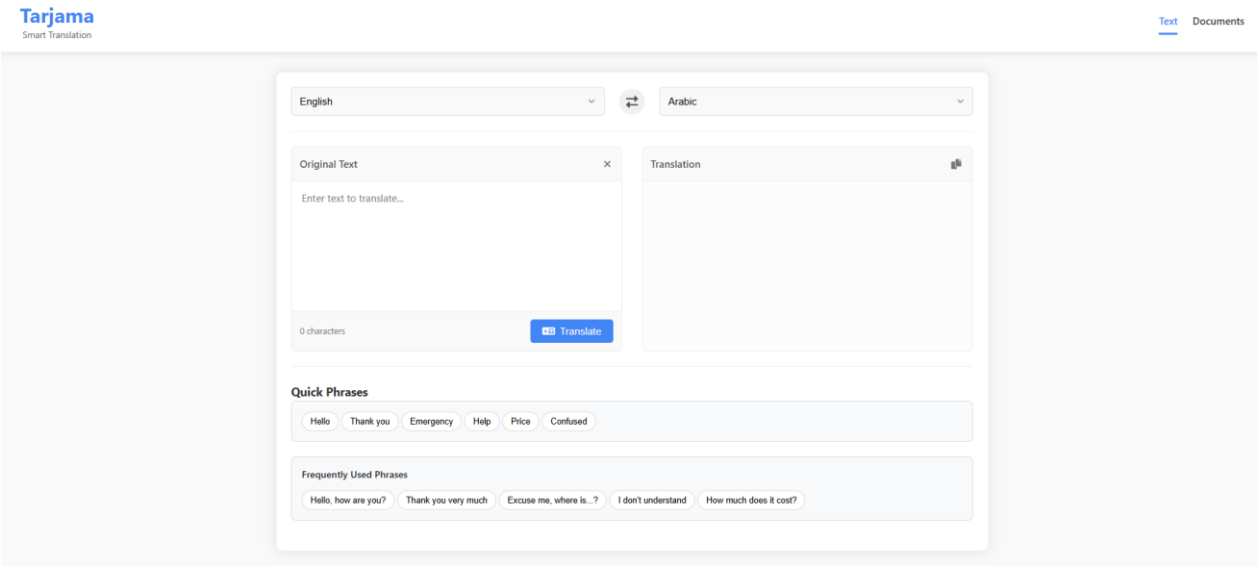
- **Clean Interface**: Minimalist design with a focus on functionality and ease of use (inspired from reverso and google translate).
- **Tabbed Navigation**: Clear separation between text translation and document translation sections.
- **Responsive Design**: Adapts to different screen sizes using CSS media queries.
- **Material Design Influence**: Uses card-based UI components with subtle shadows and clear visual hierarchy.
- **Color Scheme**: Professional blue-based color palette with accent colors for interactive elements.
- **Accessibility**: Appropriate contrast ratios and labeled form elements.

5.2. UI Components and Features

5.2.1. Text Translation Interface

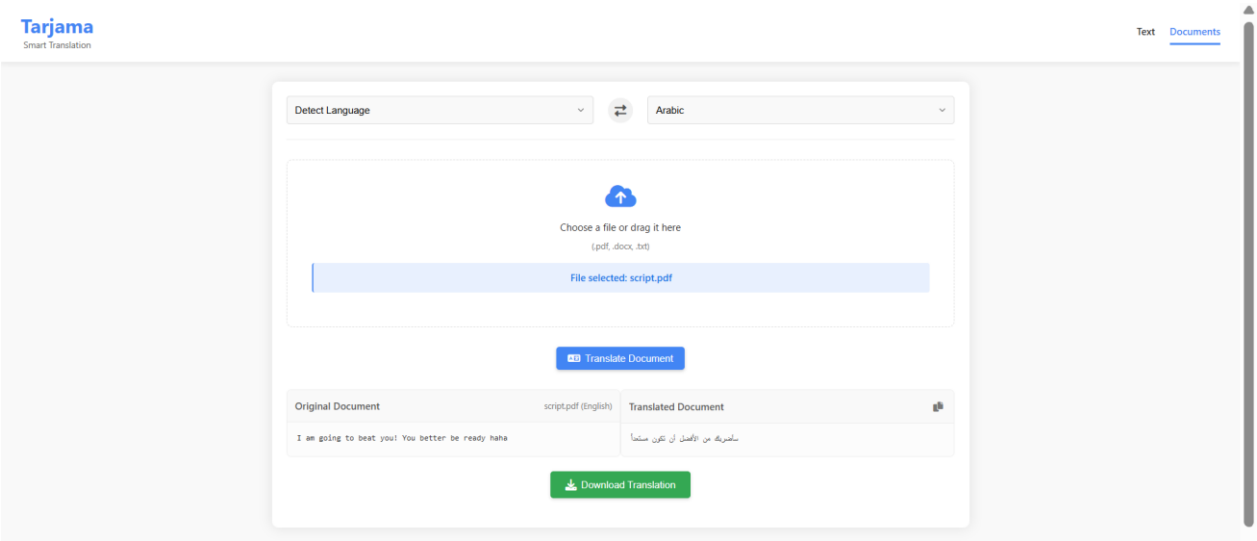
- **Language Controls**: Intuitive source and target language selectors with support for 12+ languages.
- **Language Swap Button**: Allows instant swapping of source and target languages with content reversal.
- **Character Count**: Real-time character counting with visual indicators when approaching limits.
- **Quick Phrases**: Two sets of pre-defined phrases for common translation needs:
  - **Quick Phrases**: Common greetings and emergency phrases with auto-translate option.
  - **Frequently Used Phrases**: Longer, more contextual expressions.
- **Copy Button**: One-click copying of translation results to clipboard.
- **Clear Button**: Quick removal of source text and translation results.

- **RTL Support**: Automatic right-to-left text direction for Arabic and Hebrew.



5.2.2. Document Translation Interface

- **Drag-and-Drop Upload**: Intuitive file upload with highlighting on drag-over.
- **File Type Restrictions**: Clear indication of supported document formats.
- **Upload Notification**: Visual confirmation when a document is successfully uploaded.
- **Button State Management**: Translation button changes appearance when a file is ready to translate.
- **Side-by-Side Results**: Original and translated document content displayed in parallel panels.
- **Download Functionality**: Button to download the translated document in the original format.



5.2.3. Notification System

- **Success Notifications**: Temporary toast notifications for successful operations.
- **Error Messages**: Clear error display with specific guidance on how to resolve issues.
- **Loading Indicators**: Spinner animations for translation processes with contextual messages.

5.3. Frontend JavaScript Architecture

5.3.1. Event-Driven Design

The frontend uses an event-driven architecture with clearly separated concerns:

```
// UI Element Selection
const textTabLink = document.querySelector('nav ul li a[href="#text-
translation"]');
const textInput = document.getElementById('text-input');
const phraseButtons = document.querySelectorAll('.phrase-btn');
const swapLanguages = document.getElementById('swap-languages');

// Event Listeners
textTabLink.addEventListener('click', switchToTextTab);
textInput.addEventListener('input', updateCharacterCount);
phraseButtons.forEach(button => button.addEventListener('click',
insertQuickPhrase));
swapLanguages.addEventListener('click', swapLanguagesHandler);

// Feature Implementations
function swapLanguagesHandler(e) {
    // Language swap logic
    const sourceValue = sourceLangText.value;
    const targetValue = targetLangText.value;

    // Don't swap if using auto-detect
    if (sourceValue === 'auto') {
        showNotification('Cannot swap when source language is set to auto-
detect.');
        return;
    }

    // Swap the values and text content
    sourceLangText.value = targetValue;
    targetLangText.value = sourceValue;

    if (textOutput.textContent.trim() !== '') {
        textInput.value = textOutput.textContent;
        textTranslationForm.dispatchEvent(new Event('submit'));
    }
}
```

5.3.2. API Interaction

All API calls use the Fetch API with proper error handling:

```
fetch('/translate/text', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
        text: text,
        source_lang: sourceLang,
        target_lang: targetLang
    }),
})
.then(response => {
    if (!response.ok) {
        throw new Error(`HTTP error! Status: ${response.status}`);
    }
    return response.json();
})
.then(data => {
    // Process successful response
})
.catch(error => {
    // Error handling
    showError(`Translation error: ${error.message}`);
});
```

5.3.3. Document Download Implementation

The document download functionality uses a combination of client-side and server-side processing:

```
function downloadTranslatedDocument(content, fileName, fileType) {
    // Determine file extension
    let extension = fileName.endsWith('.pdf') ? '.pdf' :
                    fileName.endsWith('.docx') ? '.docx' : '.txt';

    // Create translated filename
    const baseName = fileName.substring(0, fileName.lastIndexOf('.'));
    const translatedFileName = `${baseName}_translated${extension}`;

    if (extension === '.txt') {
        // Direct browser download for text files
        const blob = new Blob([content], { type: 'text/plain' });
        const url = URL.createObjectURL(blob);
        triggerDownload(url, translatedFileName);
    } else {
        // Server-side processing for complex formats
        fetch('/download/translated-document', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
```

```
            body: JSON.stringify({
                content: content,
                filename: translatedFileName,
                original_type: fileType
            }),
        })
        .then(response => response.blob())
        .then(blob => {
            const url = URL.createObjectURL(blob);
            triggerDownload(url, translatedFileName);
        });
    }
}

function triggerDownload(url, filename) {
    const a = document.createElement('a');
    a.href = url;
    a.download = filename;
    document.body.appendChild(a);
    a.click();
    document.body.removeChild(a);
    URL.revokeObjectURL(url);
}
```

6. Deployment and Scalability

6.1. Dockerization

- **Base Image:** Uses an official `python:3.12-slim` image for a smaller footprint.
- **Dependency Management:** Copies `requirements.txt` and installs dependencies early to leverage Docker caching.
- **Code Copying:** Copies the necessary application code (`backend`, `templates`, `static`) into the container.
- **Directory Creation:** Ensures necessary directories (`templates`, `static`, `uploads`) exist within the container.
- **Port Exposure:** Exposes port 8000 (used by `uvicorn`).
- **Entrypoint:** Uses `uvicorn` to run the FastAPI application (`backend.main:app`), making it accessible on `0.0.0.0`.

6.2. Hugging Face Spaces Deployment

- **Method:** Uses the Docker Space SDK option.
- **Configuration:** Requires creating a `README.md` file in the repository root with specific Hugging Face metadata (e.g., `sdk: docker`, `app_port: 8000`).
- **Repository:** The project code (including the `Dockerfile` and the `README.md` with HF metadata) needs to be pushed to a Hugging Face Hub repository (either model or space repo).
- **Build Process:** Hugging Face Spaces automatically builds the Docker image from the `Dockerfile` in the repository and runs the container.

6.3. Resource Optimization

- **Model Caching:** Translation models are stored in a writable cache directory (/tmp/transformers_cache).
- **Memory Management:** Models use PyTorch's low_cpu_mem_usage option to reduce memory footprint.
- **Device Placement:** Automatic detection of available hardware (CPU/GPU) with appropriate device placement.
- **Concurrent Execution:** Uses ThreadPoolExecutor for non-blocking model inference with timeouts.
- **Initialization Cooldown:** Implements a cooldown period between initialization attempts to prevent resource exhaustion.

6.4. Reliability Mechanisms

- **Error Recovery:** Automatic detection and recovery from model failures.
- **Model Testing:** Validation of loaded models with test translations before use.
- **Timeouts:** Inference timeouts to prevent hanging on problematic inputs.

7. Debugging and Technical Challenges

7.1. Frontend Debugging

7.1.1. Quick Phrases Functionality

Initial implementation of quick phrases had issues with event propagation and tab switching:

Problem: Quick phrase buttons weren't consistently routing to the text tab or inserting content. **Solution:** Added explicit logging and fixed event handling to ensure: - Tab switching works properly with proper class manipulation - Text insertion considers cursor position correctly - Event bubbling is properly managed

7.1.2. Language Swap Issues

The language swap functionality had several edge cases that needed handling:

Problem: Swap button didn't properly handle the "auto" language option and didn't consistently apply RTL styling. **Solution:** Added conditional logic to prevent swapping when source language is set to "auto" and ensured RTL styling is consistently applied after swapping.

7.1.3. File Upload Visual Feedback

Problem: Users weren't getting clear visual feedback when files were uploaded. **Solution:** Added a styled notification system and enhanced the file name display with borders and background colors to make successful uploads more noticeable.

7.2. Backend Challenges

7.2.1. Model Loading Failures

Problem: Translation models sometimes failed to initialize in the deployment environment. **Solution:** Implemented a multi-tier fallback system that: - Attempts model initialization with appropriate error handling - Falls back to online translation services when local models fail - Implements a cooldown period between initialization attempts

```
def initialize_model(language_pair: str):
    # If we've exceeded maximum attempts and cooldown hasn't passed
    if (model_initialization_attempts >= max_model_initialization_attempts
and
        current_time - last_initialization_attempt <
initialization_cooldown):
        return False

    try:
        # Model initialization code with explicit error handling
        tokenizer = AutoTokenizer.from_pretrained(
            model_name,
            cache_dir="/tmp/transformers_cache",
            use_fast=True,
            local_files_only=False
        )
        # ... more initialization code
    except Exception as e:
        print(f"Error loading model for {language_pair}: {e}")
        return False
```

7.2.2. Document Processing

Problem: Different document formats and encodings caused inconsistent text extraction. **Solution:** Implemented format-specific handling with fallbacks for encoding detection:

```
if file_extension == '.txt':
    try:
        extracted_text = content.decode('utf-8')
    except UnicodeDecodeError:
        # Try other common encodings
        for encoding in ['latin-1', 'cp1252', 'utf-16']:
            try:
                extracted_text = content.decode(encoding);
                break
            except UnicodeDecodeError:
                continue
```

7.2.3. Translation Download Formats

Problem: Generating proper document formats for download from translated text. **Solution:** Created format-specific document generation functions that properly handle: -

PDF creation with PyMuPDF - DOCX creation with python-docx - Proper MIME types and headers for browser downloads

7.3. Integration Testing

7.3.1. End-to-End Translation Flow

Extensive testing was performed to ensure the complete translation flow worked across different scenarios: - Text translation with various language combinations - Document upload and translation with different file formats - Error scenarios (network failures, invalid inputs) - Download functionality for different file types

7.3.2. Cross-Browser Testing

The application was tested across multiple browsers to ensure consistent behavior: - Chrome - Firefox - Safari - Edge

8. Future Work

8.1. Feature Enhancements

- **Translation Memory:** Implement translation memory to avoid re-translating previously translated segments.
- **Terminology Management:** Allow users to define and maintain custom terminology for consistent translations.
- **Batch Processing:** Enable translation of multiple documents in a single operation.
- **User Accounts:** Add authentication to allow users to save and manage their translation history.
- **Additional File Formats:** Extend support to handle more document types (PPTX, XLSX, HTML).
- **Dialect Support:** Add support for different Arabic dialects beyond Modern Standard Arabic.
- **API Documentation:** Implement Swagger/OpenAPI documentation for the backend API.

8.2. Technical Improvements

- **State Management:** Implement a more robust frontend state management solution for complex interactions.
- **Progressive Web App:** Convert the application to a PWA for offline capabilities.
- **Unit Testing:** Add comprehensive unit tests for both frontend and backend code.
- **Model Fine-tuning:** Fine-tune translation models specifically for Arabic eloquence.
- **Web Workers:** Use web workers for client-side processing of large text translations.
- **Performance Optimization:** Implement caching and lazy loading for better performance.

9. Conclusion

The Tarjama translation application successfully meets its core objectives of providing high-quality translations between multiple languages with a focus on Arabic eloquence. The implementation features a robust backend with multiple fallback systems, a user-friendly frontend with intuitive interactions, and comprehensive document handling capabilities.

Key achievements include: - Implementation of a reliable multi-model translation system - Robust fallback mechanisms ensuring service availability - Intuitive UI for both text and document translation - Support for language switching and bidirectional translation - Document upload, translation, and download in multiple formats - Quick phrase functionality for common translation needs

The application demonstrates how modern web technologies and AI models can be combined to create practical, user-friendly language tools that respect cultural nuances and focus on natural, eloquent translations.